1 The digital image

A digital image is a discrete representation of a continuous 2D function $I(x, y)$. In sensors the analog digital has to be converted by an ADC (analog to digital converter). Some challenges with images: transmission interference, compression artifacts, spilling, scratches, sensor noise, low contrast or resolution, motion blur.

Charge coupled device (CCD)

An array of photosites (a bucket of electrical charge) hold charge proportional to incident light intensity during exposure. Charges move down through sensor array, ADC (analog-digital-converter) measures line-by-line.

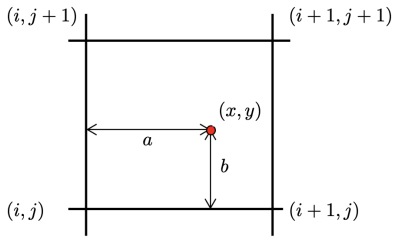
- **Blooming:** Oversaturated photosites cause vertical channels to flood (bright vertical line)
- **Bleeding/Smearing:** While charge in transit, bright light hits photosites above (worse with shorter shutter times, electrical shutters only)
- **Dark current:** Thermally generated charge (random noise despite darkness), avoid by cooling; worsens with age
- High **production cost** and **power consumption**

CMOS sensors

Mostly same as CCD, but each sensor has amplifier. Cheaper, lower power, no blooming, on-chip integration with other components, but less sensitive and more noise. Rolling shutter is an issue for sequential read-out of lines. Also susceptible to dark current.

During exposure the photons are accumulated in the sensor wells, with the accumulated charge being transferred line-by-line to the sense amplifiers which then pass the signal to the ADC. In CMOS sensors each pixel has its own amplifier and ADC.
The pinhole camera takes picture through a small hole, but cant be infinitly small, there will always be blur, so we use a use lense.
CDD is the more mature technology, with higher production cost, power consumsion, problem lies in Blooming and linearly readout. CMOS is the more recent technology that uses the same sensor as CDD, cheap, low power, less sensetiv, but enables smart pixels via other components on chip. Random accsess! We can sample an image (in 2D) in a cartesian grid, hexagonally, or in other non-uniform ways. **Undersampling** means loss of information and introduces aliasing: signals reconstructed incorrectly and higher frequencies incorrectly show up as lower frequency. If we're sampling a signal with a frequency f_s we can only uniquely capture signals with frequencies up to $f_s/2$ (Nyquist frequency). To reconstruct a continuous signal from the sample we could use biliner iterpolation:

$$\hat{f}(x,y) = (1-a)(1-b)F_{i,j} + a(1-b)F_{i+1,j} + abF_{i+1,j+1} + (1-a)bF_{i,j+1}$$



sampeling: how often we sample,
Quantanizaion: how many bits per sample (color depth).
Geometric resolution: number of pixels per area.
Radiometric resolution: number of bits per pixel.

1.1 Noise (σ)

- **Additive Gaussian Noise:** This assumes that the noise follows a normal distribution, i.e. $\tilde{F} - F = \sigma \sim \mathcal{N}(0, \tilde{\sigma}^2)$
- **Multiplicative Noise:** Here, the noise follows the intensity of the original signal: $\tilde{F} - F = \sigma = F * c, \quad c \sim \mathcal{N}(b, \tilde{\sigma}^2)$
- **Poisson noise (shot noise):** To model scene-dependent noise, the poisson distribution can be used: $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$, where λ is the signal average. Note that for large averages, the poisson distribution is similar to a gaussian distribution.
- **Rician noise (MRI):** $p(I) = \frac{1}{\sigma^2} \exp\left(-\frac{I^2+f^2}{2\sigma^2}\right) I_0\left(\frac{If}{\sigma^2}\right)$
- **Salt-and-Pepper-Noise:** Random black and white pixels are typically caused by faulty sensors or memory errors, hard to describe mathematically, easily removed with median filter.

There are a number of statistics that aim to quantify noise over an image. Among these, the most popular statistics are:

- **Signal-to-Noise-Ratio (SNR):** $\sigma_{SNR} = \frac{E[F]}{E[\sigma]}$. Because of its wide dynamic range, it is often reported in decibel (dB).
- **Peak Signal-to-Noise-Ratio (PSNR):** $\sigma_{SNR} = \frac{F_{max}}{E[\sigma]}$. For most images, F_{max} is 255.

Color camera concepts: Prism (splitting light, 3 sensors, requires good alignment), Filter mosaic (coat filter on sensor, grid, introduces aliasing), Filter wheel (rotate filter in front of lens, static image), CMOS sensor (absorbing colors at different depths), Rolling shutter effect (sequential readout of pixels while camera is moving).

2 Image Segmentation

Classicfication problem where we partition image into disjoint segments. **Complete segmentation** of I: regions $R_1, \ldots, R_N$ s.t. $I = \bigcup_{i=1}^{N} R_i$ and $R_i \cap R_j = \emptyset$ for $i \neq j$.

2.1 Thresholding

Choose a threshold T and classify pixels as foreground or background depending on whether their intensity is above or below T. For color images, use distance in color space instead of intensity to gage similarity.

$$B(x,y) = \begin{cases} 1 & \text{if } I(x,y) > T \\ 0 & \text{else} \end{cases} \quad \text{(Grayscale)}$$

$$B(x,y) = \begin{cases} 1 & \text{if } \|(I(x,y),(r,g,b)_{\text{ref}})\|_2 > T \\ 0 & \text{else} \end{cases} \quad \text{(Color)}$$

Instead of using a fixed threshold, we can model the background pixel values as a Gaussian distribution from a set of reference points $c^i_{\text{ref}} \in \mathbb{N}^3$ and then using the Mahalanobis distance to classify pixels:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} c^i_{\text{ref}} \quad \Sigma = \frac{1}{N-1}\sum_{i=1}^{N}(c^i_{\text{ref}} - \mu)(c^i_{\text{ref}} - \mu)^T$$

$$B(x,y) = \begin{cases} 1 & \text{if } (I(x,y)-\mu)^T\Sigma^{-1}(I(x,y)-\mu) > T, \\ 0 & \text{else} \end{cases}$$

Chromakeying: choose a special background color $\mathbf{x}$, then segmentation using $I_\alpha = \|\mathbf{I} - \mathbf{x}\| > T$. Issues: hard α-mask, variation not same in 3 channels.

Receiver operating characteristic (ROC)

ROC curve describes performance of binary classifier. Plots (y-axis, sensitivity, TP/(TP + FN), TPR) against (x-axis, 1-specificity, FP/(FP + TN), FPR). We can choose operating point with gradient $\beta = \frac{N}{P} \cdot \frac{V_{\text{TN}} + C_{\text{FP}}}{V_{\text{TP}} + C_{\text{FN}}}$ where V is value and C cost.

As with any classification problem, we can evaluate it using ROC curves. In the script the positive class is the foreground.

$$\text{TPR(Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

2.2 Connected Component Labeling

Connected components are regions in an image where each pixel is connected with paths where each consecutive pixel pair is a neighbour and satisfies some inclusion criteria. The 4-neighbourhood of a pixel are the pixels directly above, below, left and right of it. The 8-neighbourhood also includes the diagonal pixels.

2.3 Region Growing

Iterative process that considers the neighbours of a set of seed pixels and adds them to the region if they satisfy some criteria.

- Select seed point or initial region
- Add neighboring pixels that meet the criteria for inclusion

- Repeat the process until no more pixels can be added

For instance, after each iteration, you can calculate the region's mean and standard deviation, adding a new pixel only if it's mean normalized intensity falls within a specified range of the standard deviation.

2.4 Clustering

We can group pixels in different ways, like intensity similarity, color similarity, texture similarity, intensity+position similarity. Detects sperical clusters only.

2.5 Snakes

Active contour (polygon) where each point on contour moves away from seed while inclusion criteria in neighborhood is met, often with smoothness constraint. Iteratively minimize energy $E = E_{\text{tension}} + E_{\text{stiffness}} + E_{\text{image}}$.

2.6 Distance measures I_α

- Plain BG subtraction: $\mathbf{I}_\alpha = |\mathbf{I} - \mathbf{I}_{\text{bg}}|$
- **Mahalanobis:** $\mathbf{I}_\alpha = \sqrt{(\mathbf{I}-\mu)^T \Sigma^{-1} (\mathbf{I}-\mu)}$ where Σ is the BG image's covariance matrix: $\Sigma_{ij} = \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)^T]$, estimate from $n \geq 3$ data points: $\frac{1}{n-1}\sum_{i=1}^{n}(x_i-\mu)(x_i-\mu)^T$.

Needs thresholds: $\mathbf{I}_\alpha > \mathbf{T}$ ($T \in \mathbb{R}^3$ RGB, gray $\mathbb{R}$).

2.7 Markov Random Fields

Also want to enforce region constrains like spacial consistency and smooth borders. To achieve this we encode the cost of flipping labels as well as dependencies between pairs of pixels. We're trying to find the Assignment $\mathbf{y} \in \mathbb{N}^{W \times H}$ based on a learnable paramenter θ and some data (e.g. image) with the highest probability:

$$\arg\max P(\mathbf{y};\theta,\text{data}) = \frac{1}{Z}\prod_{i=1}^{N} p_1(y_i;\theta,\text{data}) \prod_{i,j\in\text{edges}} p_2(y_i,y_j;\theta,\text{data})$$

$$= \arg\min -\log P(\mathbf{y};\theta,\text{data})$$

$$= \arg\min \sum_i \psi_1(y_i;\theta,\text{data}) + \sum_{i,j\in\text{edges}} \psi_2(y_i,y_j;\theta,\text{data})$$

ψ_1 cost of assigning a label to each pixel, ψ_2 cost of assigning a pair of labels to connected pixels.

Example

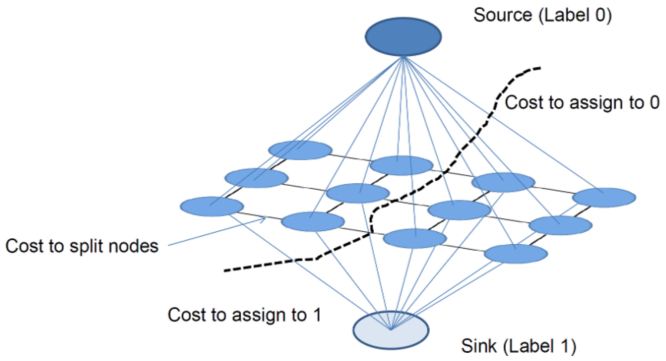
If we want to assign all pixels the label l, the potentials could be:

$$\psi_1^l(y_i;\theta,data) = -\log p(y_i = l|\theta,data)$$

$$\psi_2^l(y_i,y_j;\theta,data) = \begin{cases} 0 & \text{if } y_i = y_j \\ K & \text{else} \end{cases}$$

K is a regularization term that penalizes different labels between connected pixels.

This can be modeled as a graph min-cut problem as follows:



Solve with graph cuts, source = FG, sink = BG. Minimize energy in polynomial time using MinCut. To get decent results, we need **alpha estimation / border matting** along edges.

3 Convolution & Correlation

Linear, shift-invariant, associative,commutative (if dimensions identical). Shift invariant means that the same operation is applied at each pixel. When treating image region and kernel as vector and they're both roughly the same length then kernel-region product is high when they're similar (!template matching!). Convolution is the same as correlation but the kernel is punktgespiegelt in the center. For continuous and discrete signals, convolution is defined as follows:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

$$K * I = \sum_{i=-k}^{k}\sum_{j=-k}^{k} K(-i,-j)\cdot I(x+i,y+j),$$

Important kernels

Mean: $\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$ **Gaussian:** $\frac{1}{16}\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix}$

Laplacian: $\begin{bmatrix}0&1&0\\1&-4&1\\0&1&0\end{bmatrix}$ or $\begin{bmatrix}1&1&1\\1&-8&1\\1&1&1\end{bmatrix}$

Sobel x: $\begin{bmatrix}-1&0&1\\-2&0&2\\-1&0&1\end{bmatrix}$ **Sobel y:** $\begin{bmatrix}-1&-2&-1\\0&0&0\\1&2&1\end{bmatrix}$

Prewitt x: $\begin{bmatrix}-1&0&1\\-1&0&1\\-1&0&1\end{bmatrix}$ **Prewitt y:** $\begin{bmatrix}-1&-1&-1\\0&0&0\\1&1&1\end{bmatrix}$

diff x: $\begin{bmatrix}-1&1\end{bmatrix}$ **diff y:** $\begin{bmatrix}-1\\1\end{bmatrix}$ **High pass:** $\begin{bmatrix}-1&-1&-1\\-1&8&-1\\-1&-1&-1\end{bmatrix}$

Gaussian pyramid: repeatedly convolve image with Gaussian kernel and downsample by factor 2. Integrated image: as boxfilter requiere a lot of additions, optimize using integral image where each pixel holds the sum of all pixels above and to the left.

Image Gradient: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]^T$

Gradient direction: $\theta = \tan^{-1}(\frac{\partial f/\partial y}{\partial f/\partial x})$

Gradient magnituted (edge strength): $|\nabla f| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$

Bc of Noise the edges are not well defined, so we smooth the image first with a Gaussian and then compute the gradient (convolution theroem, we can just ableiten the kernel and then convolve with image $\partial/\partial x(f * g) = f * \partial g/\partial x$).

4 Image Filtering

Modify pixels on some function of local neighb. **Shift-invariant**: same for all pixels, K not dependent on pos.
Linear: linear combination of neighb.
Local: output only depends on pixels in its neighbors (not global).

Correlation

Template matching: $I' = K \circ I$ $I(x,y) = \sum_{(i,j)\in N(x,y)} K(i,j)I(x+i,y+j)$

Convolution

Point spread function: $I' = K * I$ $I'(x,y) = \sum_{(i,j)\in N(x,y)} K(i,j)I(x-i,y-j)$ Linear, associative, shift-invariant, commutative (if dimensions are identical). For the continuous case: $g(x) = f(x) * k(x) = \int_{\mathbb{R}} f(a)k(x-a)\,da$

The 2 operations are the same with reversed kernels.
Filtering near edges: clip to black, wrap around, copy edge, reflect across edge, vary filter.
Separable: kernel $K(m,n) = f(m)g(n)$, if the kernel matrix has rank 1 it's separable because it's the outer product of two vectors. The Gaussian kernel is rot. symmetric, single lobe (neighbor's influence decreases monotonically, also in frequency domain), FT is again a Gaussian, separable, easily made efficient.
Laplacian is rot. invariant (isotropic), usually more noisy since it uses 2nd derivative.
Image sharpening: enhances edges by increasing high frequency components: $I' = I + \alpha|k * I|$ where k high-pass filter, $\alpha \in [0,1]$.

5 Edge Detection

Laplacian operator

Find 0s in 2nd deriv I'' to locate edges. Rot. invariant, yields very noisy but thin and uninterrupted edges. Sensitive, blur first (Laplacian of Gaussian) or suppress edges with low gradient magnitude.

$$\mathrm{LoG}(x,y) = -\frac{1}{\pi\sigma^4}\left(1-\frac{x^2+y^2}{2\sigma^2}\right)\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

$$\nabla^2 f(x,y) = \frac{\partial^2 f(x,y)}{\partial x^2} + \frac{\partial^2 f(x,y)}{\partial y^2}$$

DoG can approximate LoG.

Canny edge detector

Thin, uninterrupted edges, no guarantee on the connectivity of edges, extended more completely than with simple thresh.

- Smooth image with Gaussian filter
- Compute grad. mag. and orient. (Sobel, Prewitt, ...): $M(x,y) = \sqrt{(\partial f/\partial x)^2 + (\partial f/\partial y)^2}$, $\alpha(x,y) = \tan^{-1}\left(\frac{\partial f/\partial y}{\partial f/\partial x}\right)$
- Nonmaxima suppression: quantize edges normal to 4 dirs, if smaller than either neighb. suppress
- Double thresholding (hysteresis): $T_{\text{high}}, T_{\text{low}}$, keep if $\geq T_{\text{high}}$ or $\geq T_{\text{low}}$ and 8-conn. through $\geq T_{\text{low}}$ to T_{high} px.

Hough transformation

Fits a curve (line, circles, ...) to set of edge pixels (e.g. $y = mx + c$)

- Subdivide (m,c) plane into discrete bins init to 0
- Draw line in (m,c) plane for each edge pixel (x,y), increment bins by 1 along line
- Detect peaks in (m,c) plane.

Infinite slopes arise, reparameterize line with (θ,ρ): $x\cos\theta + y\sin\theta = \rho$. For circles with known radius: $(x-a)^2+(y-b)^2 = r^2$, else 3D Hough.

Corner detection: Define local displacement sensitivity: $S(\Delta x,\Delta y) = \sum_{(x,y)\in\text{window}}(f(x,y) - f(x+\Delta x, y+\Delta y))^2$. Using Taylor approximation and $\mathbf{M}$:

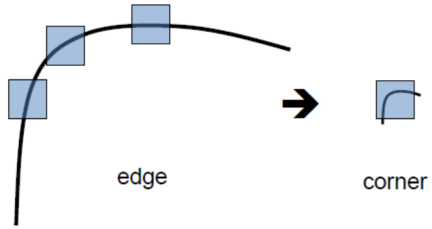
$$f(x+\Delta x, y+\Delta y) \approx f(x,y) + f_x(x,y)\Delta x + f_y(x,y)\Delta y$$

$$\mathbf{M} = \sum_{(x,y)\in\text{window}} \begin{bmatrix} f_x^2(x,y) & f_x(x,y)f_y(x,y) \\ f_x(x,y)f_y(x,y) & f_y^2(x,y) \end{bmatrix}$$

$$S(\Delta x,\Delta y) \approx \begin{bmatrix}\Delta x & \Delta y\end{bmatrix}\mathbf{M}\begin{bmatrix}\Delta x \\ \Delta y\end{bmatrix}$$

Harris corner detection

Compute matrix $\mathbf{M}$. Compute $C(x,y) = \det(\mathbf{M}) - k\,\mathrm{trace}(\mathbf{M})^2 = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2$. Mark as corner if $C(x,y) > T$. Do non max-suppression and for better localization, weight central pixels with weights for sum in $\mathbf{M}$: $G(x-x_0, y-y_0, \sigma)$. Compute subpixel localization by fitting parabola to cornerness function.



Can be made **scale-invariant** by looking for strong responses to **DoG filter** over scale space, then consider loc. max. in both position and scale space (see SIFT).

Lowe's Scale Invariant Feature Transform (SIFT)

Used to track feature points over 2 images. Look for strong responses in Difference of Gaussian (DoG) over scale space and position, consider local maxima in both spaces to find blobs. Compute histogram of gradient directions (ignoring gradient mag. bc lighting etc.) at selected scale, pos., rot. by choosing principal direction. Now both pictures are at the same scale & orientation, compare gradient histog. to find matching points. $\mathrm{DoG}(x,y) = \frac{1}{k}e^{-(x^2+y^2)/(k\sigma)^2} - e^{-(x^2+y^2)/(\sigma^2)}$, $k = \sqrt{2}$.

6 Fourier Transform

$$F(u) = \int_{-\infty}^{\infty} f(x)e^{-i2\pi ux}dx,$$

$$f(x) = \int_{-\infty}^{\infty} F(u)e^{i2\pi ux}du$$

$$e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$$

F(u) holds the amplitude and phase of the corresponding sinusoid.
Amplitude/Magnitude: $A(u) = |F(u)| = \sqrt{(\Re\{F(u)\})^2 + (\Im\{F(u)\})^2}$.
Phase:$\phi(u) = \arg F(u) = \tan^{-1}\left(\frac{\Im\{F(u)\}}{\Re\{F(u)\}}\right)$

$$F(u,v) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(x,y)e^{-i2\pi(ux+vy)}dx\,dy,$$

$$f(x,y) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} F(u,v)e^{i2\pi(ux+vy)}du\,dv$$

2d basis function:

$$e^{i2\pi(ux+vy)} = \cos(2\pi(ux+vy)) + i\sin(2\pi(ux+vy))$$

Basis function has maximal real value when $ux + vy \in \mathbb{N}$. Places where this holds form a line in 2d space characterized by normal vector (u,v). The distance between two lines (wavelength) is $\frac{1}{\sqrt{u^2+v^2}}$, frequency is inverse. u and v determine frequency and orientation of the basis function. Pluggin u,v into the Fourier transform of the functiion gives a complex number from which the phase and amplitude can be extracted.

- **Magnitude spectrum** tells you how much of each frequency is present (energy distribution).
- **Phase spectrum** tells you how those frequencies are aligned in space, and this is what gives an image its actual structure.

Discrete FT: $F = \mathbf{U}f$ where F transformed image, $\mathbf{U}$ FT base, f vectorized image. $F(u,v) = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f(x,y)\cdot\exp\left(-i2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)\right)$,
Dual Transform: $f(-x) = F(F(f))(x)$. **Relevant:** $\cos(x) = \frac{e^{ix}+e^{-ix}}{2}$, $\sin(x) = \frac{e^{ix}-e^{-ix}}{2i}$, $\mathrm{sinc}(u) = \frac{\sin(u)}{u}$, $\int_{-\infty}^{\infty} e^{-2\pi ixu}\,dx = \delta(u)$. **Dirac delta:** $\delta(x) = 0$ if $x \neq 0$ else undefined.

- $\int_{-\infty}^{\infty}\delta(x)\,dx = 1$, $\delta(\alpha x) = \delta(x)/|\alpha|$ and $\delta(-t) = \delta(t)$

- $(\delta * f)(x) = \int_{-\infty}^{\infty} f(t)\delta(x-t)\,dt = f(x)$, e.g. $\delta(u-k) * f(u) = f(u-k)$

Sampling: multiplication with a sequence of δ-functions, **Fourier Rotation Theorem**: $F[f \circ R] = F[f] \circ R$ for rotation matrix R. **Convolution theorem**:

$$f(x,y) * h(x,y) \iff F(u,v) \cdot H(u,v) \text{((element wise))}$$

Property	$\mathbf{f(x)}, f(x,y)$	$\mathbf{F(u)}, F(u,v)$
Linearity	$\alpha f_1(x) + \beta f_2(x)$	$\alpha F_1(u) + \beta F_2(u)$
Duality	$F(x)$	$f(-u)$
Convolution	$(f * g)(x)$	$F(u) \cdot G(u)$
Product	$f(x)g(x)$	$(F * G)(u)$
Timeshift	$f(x - x_0)$	$e^{-2\pi i u x_0} \cdot F(u)$
Freq. shift	$e^{2\pi i u_0 x} f(x)$	$F(u - u_0)$
Differentiation	$\frac{d^n}{dx^n} f(x)$	$\left(\frac{i}{2\pi} u\right)^n F(u)$
Multiplication	$xf(x)$	$\frac{i}{2\pi} \frac{d}{du} F(u)$
Stretching	$f(ax)$	$\frac{1}{\lvert a \rvert} F\left(\frac{u}{a}\right)$
Deriv. Fourier	$x^n f(x)$	$i^n \frac{d^n}{du^n} F(u)$

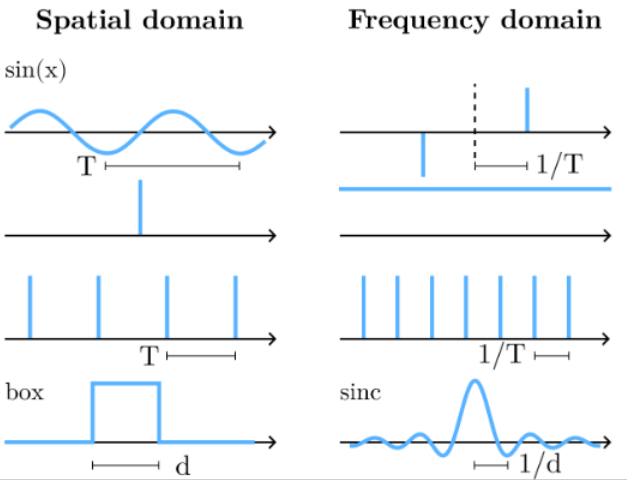


Image restoration

Image degradation is applying kernel h to some image f. The inverse $\tilde{h}$ should compensate: $f(x) \to h(x) \to g(x) \to \tilde{h}(x) \to f$. Determine with $F(\tilde{h})(u,v) = F(h)(u,v) = 1$. Cancellation of freq., noise amplif. Regularize using $\tilde{F}(\tilde{h})(u,v) = F(h)/(|F(h)|^2 + \epsilon)$.

$\mathbf{f(x)}, f(x,y)$	$\mathbf{F(u)}, F(u,v)$
$\sin(2\pi u_0 x + 2\pi v_0 y)$	$\frac{1}{2i}[\delta(u-u_0, v-v_0) - \delta(u+u_0, v+v_0)]$
$\cos(2\pi u_0 x + 2\pi v_0 y)$	$\frac{1}{2}[\delta(u-u_0, v-v_0) + \delta(u+u_0, v+v_0)]$
$\sin(2\pi u_0 x)$	$\frac{1}{2i}[\delta(u-u_0) - \delta(u+u_0)]$
$\cos(2\pi u_0 x)$	$\frac{1}{2}[\delta(u-u_0) + \delta(u+u_0)]$
1	$\delta(u)$
$\text{Box}(x) = \begin{cases} 1 & x \in [-1/2, 1/2] \\ 0 & \text{else} \end{cases}$	$\text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$ (norm. sinc)
$\text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$	$\text{Box}(x) = \begin{cases} 1 & x \in [-1/2, 1/2] \\ 0 & \text{else} \end{cases}$
$\text{sinc}(u) = \frac{\sin(u)}{u}$	$\pi \text{Box}(\pi x)$
$\sum_{n=-\infty}^{\infty} \delta(x - nT)$	$\frac{1}{T} \sum_{n=-\infty}^{\infty} \delta(u - n/T)$
$h(x,y) = f(x)g(x)$	$H(u,v) = F(u)G(v)$
$\delta(x)$	1
$\delta(x - x_0)$	$e^{-2i\pi u x_0}$
$e^{2i\pi(u_0 x + v_0 y)}$	$\delta(u - u_0, v - v_0)$
e^{-ax^2}	$\sqrt{\frac{\pi}{a}} \exp\left(-\frac{\pi^2 u^2}{a}\right)$

7 Unitary transforms (PCA / KL)

Images are vectorized row-by-row. Linear image processing algorithms can be written as $g = Ff$. Auto-correlation fun.: $R_{ff} = \mathbb{E}[f_i f_i^H] = (FF^H)/n$.

Eigenmatrix: Φ of autocorrelation matrix R_{ff}: Φ is unitary, columns form set of eigenvectors of R_{ff}: $R_{ff}\Phi = \Phi\Lambda$ where Λ is a diag. matrix of eigenvecs. R_{ff} is symmetric nonneg. definite, hence $\lambda_i \geq 0$, and normal: $R_{ff}^H R_{ff} = R_{ff} R_{ff}^H$.

Autocorrelation matrix: $\mathbb{E}[XX^\top]$, unlike covariance matrix, we do not subtract the mean here ($\mathbb{E}[(X-\mu)(X-\mu)^\top]$).

Karhunen-Loeve/(PCA)

- **Normalize** (remove brightness var.):

$$x_i' = \frac{x_i}{\|x_i\|}$$

- **Center data** (subtract mean):

$$x_i'' = x_i' - \mu, \quad \text{where } \mu = \frac{1}{N} \sum x_i'$$

- **Compute Covariance Matrix:**

$$\Sigma = \frac{1}{N-1} \sum x_i''(x_i'')^T$$

- **Eigendecomposition:** Solve $\Sigma e = \lambda e$ (e.g., via SVD: $\Sigma = U\Lambda U^T$).
- **Define U_k:** The first k eigenvectors of Σ, $U_k = [u_1, \ldots, u_k]$ (directions with largest variance).
- **Projection (Compression):**

$$\text{PCA}(x_i) = U_k^T(x_i - \mu) = U_k^T x_i''$$

- **Reconstruction:** To decompress (where y_i is the lower-dim representation):

$$\text{PCA}^{-1}(y_i) = U_k y_i + \mu$$

PCA storage space

Given n images of size $x \times y$, we want to store the dataset given a budget of Z units of space. What is max number K of princip. comp. allowed?
We need to store dataset mean $\mu : x \times y$, truncated eigenmat. $U_k : (x \times y) \times K$, compr. imgs. $y_i : n \times K$

7.1 Eigenfaces, Fisherfaces

Simple recognition, compare in projected space, find nearest neighbour. Find face by computing reconstr. error and minimizing by varying patch pos. Compress data and visualization. Eigenfaces struggle with lighting differences. Fisherfaces improve this by maximizing between-class scatter, minimzing within-class scatter.

7.2 LDA / Fisherfaces

Find directions where ratio between/within individual variance is maximized (i.e. minimizing within class difference, maximizing between-class difference).

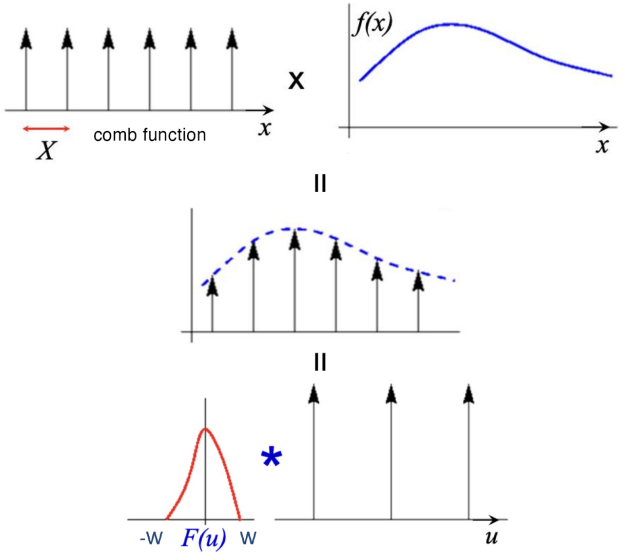
7.3 JPEG Compression

JPEG and JPG are the same format; JPG exists because older Windows versions required 3-letter extensions.

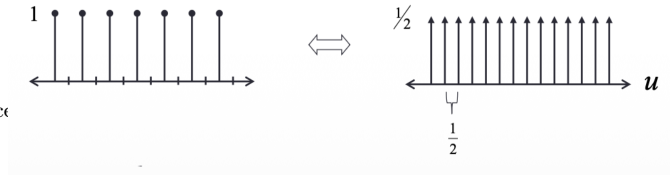
- **RGB → YUV:** Y = luminance/brightness, UV = color/chrominance. Humans more sensitive to color ⇒ compress colors with **chroma subsampling** (e.g., use color of upper-left pixel for 4×4 grid).
- **Block DCT:** Split image into 8×8 blocks for each YUV component, apply 2D DCT. Results in 64 values: top-left = low freq., bottom-right = high freq. DCT is a variant of DFT with fast implementation and only real values.
- **Quantization:** Compress using integer division with weighted matrix (quantization table), aggressively compress bottom-right (high freq.). Then zig-zag run-length encoding followed by Huffman coding.

Properties: High compression (10–100×), 24-bit colors. **Artifacts:** incorrect colors, aliasing, softened edges (sharp edges require high frequencies).
JPEG2000: Uses Haar wavelet transform globally (not just 8×8 blocks) on successively downsampled image (image pyramid).

8 Sampling



As the comb samples get further apart, the frequency samples get closer together.



Nyquist theorem

$f_{\max} \cdot 2 \leq 1/x$ = sampling rate Bc of the convolution theorem it holds: $f_{\text{comb}} \cdot f_{\text{spatial}} = f_{\text{comb}} * F(u) (= f_{\text{frequency}})$

9 Optical Flow

We assume brightness constancy. Let I(x,y,t) be the image brightness at location (x,y) at time t. Then:

$$\begin{aligned} I(x,y,t) &= I(x + \frac{dx}{dt}\delta t, y + \frac{dy}{dt}\delta t, t + \delta t) \\ &\overset{\text{taylor}}{\approx} I(x,y,t) + \frac{\partial I}{\partial x} \cdot \frac{dx}{dt}\delta t + \frac{\partial I}{\partial y} \cdot \frac{dy}{dt}\delta t + \frac{\partial I}{\partial t} \cdot \delta t \\ \implies & \frac{\partial I}{\partial x}\frac{dx}{dt} + \frac{\partial I}{\partial y}\frac{dy}{dt} + \frac{\partial I}{\partial t} = 0 \end{aligned}$$

To linearize I using Taylor expansion we assume the motion is small from frame to frame. Denoting $\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}, \frac{\partial I}{\partial t}$ as I_x, I_y, I_t respectively, and $\frac{dx}{dt}, \frac{dy}{dt}$ as u, v, the equation is commonly written as $I_x \cdot u + I_y \cdot v + I_t = 0$

9.1 Horn-Schunk Algorithm

Since neighboring pixels tend to move similarly we can add a smoothness constraint.

Global Smoothness

Let u_x denote the change of u in x direction, and so on for u_y, v_x, v_y. We then define the smoothness erro as:

$$e_s := \iint u_x^2 + u_y^2 + v_x^2 + v_y^2 dxdy$$

Denoting $e_c := \iint (I_x u + I_y v + I_t)^2 dxdy$, the algorithm focuses on minimizing the combined error $e := e_c + \lambda e_s$ where λ is a weighting factor. It is minmized using iterative methods with finite differences.

9.2 Lukas-Kanade Algorithm

This algorithm aims to enforce consistency in small patches. We assume a single velocity for all pixels in a patch Ω. And minimize: $\sum_{(x,y)\in\Omega} (I_x(x,y)u + I_y(x,y)v + I_t(x,y))^2$ whose least squares solution is given by solving the system:

$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

Sometimes there isn't a unique solution to the system, like when a patch contains only an edge moving to the right with no corner visible (gradient in one direction is zero).
The algorithm fails however if the intensity structure in the patch is insufficient (e.g. flat region, edge) or the movement is too large. To mitigate these issues the algorithm is run on increasingly finer scales of the image with the previous coarser scale's flow used as initialization for the next finer scale. **Aperture problem:** when flow is computed for point along linear feature (e.g. edge), not possible to determine exact location of corresponding point in second image, only possible to determine the flow normal to linear feature, 2 unknowns for every pixel (u, v) but only one equation ⇒ ∞ solutions, opt. flow defines a line in (u, v) space, compute normal flow. Need additional constraints to solve.

Iterative refinement

Estimate OF with Lucas-Kanade. Warp image using estimated OF. Estimate OF again using warped image. Refine estimate by repeating. Fails if intensity structure poor / large mov.

Coarse-to-fine estimation

Create levels by gradual subsampling. Start at coarsest level, estimate OF, iterate and add until reached finest level. Result is OF at finest level.

Still fails if large lighting changes happen.

Affine motion (Ax + b)

Each pixel provides 1 lin. constr., 6 global unknowns. Solve LSE. From bright. const. eq.:

$$I_x(a_1 + a_2 x + a_3 y) + I_y(a_4 + a_5 x + a_6 y) + I_t \approx 0$$

SSD tracking: Large displacements, extract template around pixel, match within search area, use correlation, choose best match. **Bayesian Optical flow:** Some low-level human motion illusions can be explained by adding an uncertainty model to LK. E.g. bright. const. with noise.

10 Video Compression

Visual perception < 24 Hz. Flicker > 60 Hz in periphery. **Bloch's law:** if stimulus duration ≤ 100 ms, duration and brightness exchangeable. If brightness is halved, double duration. This enforces > 10 Hz for videos.

Interlaced video format: 2 temporally shifted half images (in bands) increases frequency, reduces spatial resolution. Full image repr. is progressive.

Video compression with temporal redundancy

Predict current frame based on previously coded frames. Introducing 3 types of coded frames:

- I-frame: Intra-coded frame, coded independently
- P-frame: Predictively-coded based on previously coded (P and I, H.264 can also allow B) frames (e.g. motion vec. + changes)
- B-frame: Bi-directionally predicted frame, based on both previous and future frames. In older standards B-frames are never used as references for prediction of other frames.

Inefficient for many scene changes or high motion.

Motion-compensated prediction

Partition video into moving objects – generally pretty difficult. Practical: **block-matching motion est.**: partition each frame into blocks, describe motion of block / find best matching block in reference frame. No obj. det., good perf. Can be sped up with 3 step log search, and improve precision with half-pixel motions. We encode the residual error (with JPG). Motion vector field (set of motion vec. for each block in frame) is easy to encode, fast, simple, periodic.

MPEG Group of Pictures (GOP) starts with I-frame, ends with B- (open) or P-frame (closed).

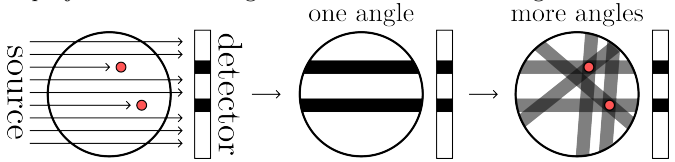
11 CNN

Given input x, learning target y, loss function $\mathcal{L}$ compute kernel weights θ using prediction $f(x,\theta)$: $\arg\min_\theta \mathcal{L}(y, f(x,\theta))$. Linear classifier $f(x,\theta) = Wx + b$ where $\theta = \{W, b\}$. Use activation func. ϕ (sigmoid, **ReLU**, tanh, ...) to introduce non-linearity. Use gradient descent and back propagation (recursive appl. of chain rule to compute gradients) to find θ. Transformers, GANs, stable diffusion etc enable modern VC breakthroughs.

Classification: $f(x_1,\theta)$ as score, take class with larger score. With $y_i \in \mathbb{N}, s_i = f(x_i,\theta)$, we get loss function: $\mathcal{L}(y, f(x,\theta)) = -\sum_{i=1}^{N} \log\left(\frac{\exp(s_{i,y_i})}{\sum_j \exp(s_{i,j})}\right)$.

Regression: $f(x_1,\theta)$ as value, can be used for classification by comparing value. With $y_i \in \mathbb{R}^n, s_i = f(x_i,\theta)$, we get: $\mathcal{L}(y, f(x,\theta)) = \sum_{i=1}^{N} \|y_i - s_i\|^2$.

Fourier Slice Theorem: $G_\theta(\omega) = F[f](\omega\cos(\theta), \omega\sin(\theta))$ where $G_\theta(\omega) = \int_{-\infty}^{\infty} R[f](\rho,\theta)\exp(-2\pi i\omega\rho)\,d\rho = F[R[f](\rho,\theta)](\omega)$, 1D FT of projection at fixed angle θ is slice of 2D FT of image.



Back projection algorithm: Given RT, find $u(x,y)$, apply for all proj. angles θ:

- Measure projection (attenuation) data, get $R[f](\rho,\theta)$ for fixed θ
- 1D FT of projection data, get $G_\theta(\omega) = F[f](\omega\cos(\theta), \omega\sin(\theta))$ as above
- 2D inverse FT the above, then sum with previous image (backpropagate)

Requires precise attn. meas., sensitive to noise, unstable, blurring in final image (add high-pass filter in Fourier domain after 2nd step to prevent).

12 General Graphics

Graphics Pipeline:

- **Modeling Transform:** Object space → world space (translate, rotate, scale)
- **Viewing Transform:** World space → camera/view space
- **Primitive Processing:** Prepare triangles, lines, points
- **3D Clipping:** Remove geometry outside view frustum
- **Projection:** 3D → 2D (perspective/orthographic)
- **Scan Conversion:** Rasterize primitives to fragments/pixels
- **Lighting/Shading/Text:** Compute fragment colors
- **Occlusion:** Z-buffer depth testing
- **Display:** Output to framebuffer

Programmer's View: Vertex Processing (per-vertex transforms, lighting) → Rasterization (interpolate attributes) → Fragment Processing (shading, texturing, blending) → Display.

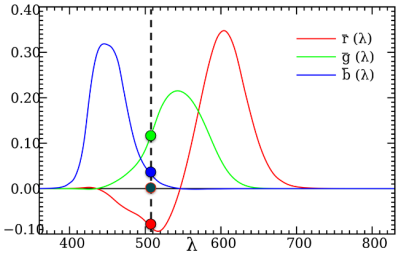
13 Light and Color

Light is a electromagnetic radiation at Frequency (wavelength $400nm$ to $700nm$). Spectral Power Distribution:
$P(\lambda)$ = intensity at wavelength λ.
Our eye has 3 types of cones (S (blue),M (green),L (red)) with different sensitivity curves. Eye projects $P(\lambda)$ into 3D subspace using these three basis functions/vectors (two diffrent SPD might look exactly the same to us).

13.1 Color spaces

- **RGB**: useful for displays, RGB colors specified
- **CIE XYZ**: change of basis to avoid negative comps. From xyY via $X = (xY)/y$, $Y = Y$, $Z = Y/y - (xY)/y - Y$
- **CIE xyY**: chromaticity (x, y) derived by normalizing XYZ: $x = X/(X+Y+Z)$, $y = Y/(X+Y+Z)$, $z = 1 - x - y$
- **CIE RGB**: (435.8, 546.1, 700.0nm). Linear combination span triangle, the color gamut
- **CMY**: inverse (subtractive) to RGB. CMY = 1 - RGB
- **YIQ**: Luminance Y, In-phase I (orange-blue), Quadrature Q (purple-green), NTSC norm
- **HSV / HLS**: hue, saturation, value/lightness; intuitive for artists
- **CIELAB / CIELUV**: perceptually uniform color space

13.2 CIE



435.8, 546.1 and $700.0nm$ (arbitrary choosen) where controlled (dimmed/increased) by observers to match a simulus color. Result is seen above, x is frequency and y is the dimmed strength that the participents did. Negativ light is not physically possible, was added bc participents could not recreate this color with just red, green and blue.

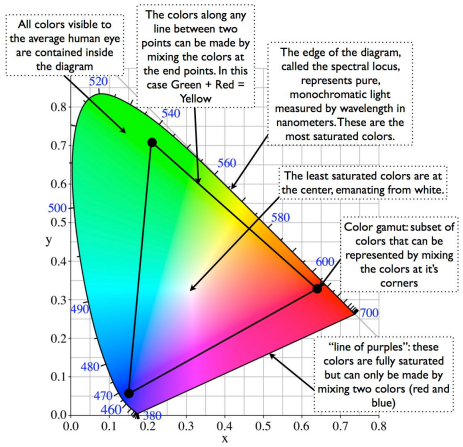
$$\text{reference} = \text{blue} + \text{green} - \text{red}$$
$$\text{reference} + \text{red} = \text{blue} + \text{green}$$

13.3 CIE-XYZ/xyY color space

choosing the $\hat{r}, \hat{g}, \hat{b}$ basis is fairly arbitrary, to avoid negative values we can define a new basis:

$$\begin{pmatrix}\bar{x}(\lambda)\\ \bar{y}(\lambda)\\ \bar{z}(\lambda)\end{pmatrix} = \begin{pmatrix}2.36 & -0.515 & 0.005\\ -0.89 & 1.426 & 0.014\\ -0.46 & 0.088 & 1.009\end{pmatrix}\begin{pmatrix}\bar{r}(\lambda)\\ \bar{g}(\lambda)\\ \bar{b}(\lambda)\end{pmatrix}$$
$$X = \int_0^\infty P(\lambda)\bar{x}(\lambda)d\lambda$$
$$Y = \int_0^\infty P(\lambda)\bar{y}(\lambda)d\lambda$$
$$Z = \int_0^\infty P(\lambda)\bar{z}(\lambda)d\lambda$$
$$x = \frac{X}{X+Y+Z}$$
$$y = \frac{Y}{X+Y+Z}$$

While Y describes brightness, x and y describe color and give us the CIE chromaticity chart:



For a given display, the phosphorous coordinates $(x_R, y_R), (x_G, y_G), (x_B, y_B)$ tell us what it understands as red, green and blue. They form the triangle above. Using these we can easilu convert from RGB to XYZ:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} x_R C_R & x_G C_G & x_B C_B \\ y_R C_R & y_G C_G & y_B C_B \\ (1-x_R-y_R)\cdot C_R & (1-x_G-y_G)\cdot C_G & (1-x_B-y_B)\cdot C_B \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The unknowns C_R, C_G, C_B result from a known white point $(X_\omega, Y_\omega, Z_\omega)$ for $1 = R = G = B$:

$$\begin{bmatrix} X_\omega \\ Y_\omega \\ Z_\omega \end{bmatrix} = \begin{bmatrix} x_R & x_G & x_B \\ y_R & y_G & y_B \\ (1-x_R-y_R) & (1-x_G-y_G) & (1-x_B-y_B) \end{bmatrix} \cdot \begin{bmatrix} C_R \\ C_G \\ C_B \end{bmatrix}$$

White point calibration

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x_R C_R & x_G C_G & x_B C_B \\ y_R C_R & y_G C_G & y_B C_B \\ (1-x_R-y_R)C_R & (1-x_G-y_G)C_G & (1-x_B-y_B)C_B \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Set $(R, G, B) = (1, 1, 1)$ and map to given white point in xyz (e.g. $(0.9505, 1, 1.0890)$), then find C_R, C_G, C_B.

13.4 CMY color space

- **Passive Systems:** Used for printers (subtractive mixing).
- **Inverse to RGB:**

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

- **CMYK:** Adds **Black (K)** to improve contrast and reduce ink usage.

13.5 YIQ Color Space

- **Components:** Luminance (Y), In-phase (I, orange-blue, Humans are very sensitive to this so it got more bandwidth), Quadrature (Q, purple-green, kind of blind in those colors).
- **Usage:** NTSC (US TV standard); advantageous for natural/skin colors.
- **RGB to YIQ Transformation:**

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

13.6 HSV & HSL/HSB Color Spaces

- **Concept:** User-oriented and intuitive (painters/designers); separates color info (H, S) from intensity (V).
- **Geometry:** Derived by projecting the RGB Cube onto a hexagon (forming a Hexcone).
- **Dimensions:**
  - **Hue (H):** The "base color" (angle on the hexagon, $0° - 360°$).
  - **Saturation (S):** Purity/Richness (radius from center; 0 = gray).
  - **Value (V):** Brightness/Lightness (vertical axis).
- **RGB → HSV Conversion:** code TODO.

13.7 CIELAB and CIELUV Color Spaces

moving on the horseshoe changes the perceived color difference non-linearly (sometimes tini movement and big change in precived color, sometimes the oposit). To fix this we can define a new color space where euclidean distances correspond to perceived color differences.

13.8 high dinamic range (HDR)

take multiple images at different exposures and combine them to get a higher dynamic range.

1. **RGB → YUV:** Y = luminance/brightness, UV = color/chrominance Humans more sensitive to brightness than color ⇒ compress colors with **chroma subsampling** (e.g., use color of upper-left pixel for 4×4 grid).
2. **Block DCT:** Split image into 8×8 blocks for each YUV component, apply 2D DCT. Results in 64 values: top-left = low freq., bottom-right = high freq. DCT is a variant of DFT with fast implementation and only real values.
3. **Quantization:** Compress using integer division with weighted matrix (quantization table), aggressively compress bottom-right (high freq.). Then zig-zag run-length encoding followed by Huffman coding.

Properties: High compression (10–100×), 24-bit colors. **Artifacts:** incorrect colors, aliasing, softened edges (sharp edges require high frequencies).
JPEG2000: Uses Haar wavelet transform globally (not just 8×8 blocks) on successively downsampled image (image pyramid).

Image Pyramids

Iteratively apply approximation filter + downsampler.

- **Gaussian pyramid:** Successively blurred and downsampled versions.
- **Laplacian pyramid:** Difference between two consecutive levels of Gaussian pyramid (stores detail/edges).

14 Transforms

Rigid Transform: Orthogonal transform + translation.
Change position & orientation of objects, project to screen, animating objects, ... **Homogeneous coordinates** can represent affine maps (translation) with mat.-mul. Add dimension, project vertices $\begin{bmatrix} x & y & z & w \end{bmatrix}^T$ onto $\begin{bmatrix} x/w & y/w & z/w & 1 \end{bmatrix}^T$.

Common transform matrices

Translation
$$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scale
$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rot. x
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

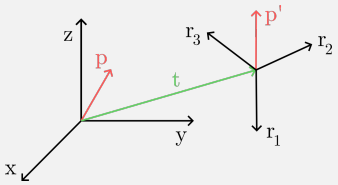
Rot. y
$$\begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rot. z
$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Shear
$$\begin{bmatrix} 1 & 0 & sh_x & 0 \\ 0 & 1 & sh_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rigid transforms: translation, rotation, (sometimes includes reflection). **Linear:** Rotation, Scaling, Shear. **Projective transforms:** Rigid + Linear + Persp. + Paral.
Commutativity ($M_1 M_2 = M_2 M_1$) holds for: (translation, translation), (rotation, rotation in 2D), (scaling, scaling), (scaling, rotation).

Change of coordinate system

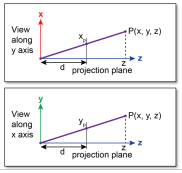


$$\mathbf{p}' = \underbrace{\begin{bmatrix} \mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t} \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\mathbf{M}} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix}$$

Transforming a normal: $\mathbf{n}' = (\mathbf{M}^{-1})^T\mathbf{n} = (\mathbf{M}^T)^{-1}\mathbf{n}$

Parallel (orthographic) projection $\mathbf{M}_{\text{ort}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Perspective projection $\mathbf{M}_{\text{per}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$



$x_p = dx/z \qquad y_p = dy/z$

$$\mathbf{M}_{per} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$$

Quaternions

Similar to $\mathbb{C}$, define $i^2 = j^2 = k^2 = -1$, $ijk = -1$, $ij = k$, $ji = -k$, $jk = i$, $kj = -i$, $ki = j$, $ik = -j$. For $q = a + bi + cj + dk$:

- $\|q\| = \sqrt{a^2 + b^2 + c^2 + d^2}$, $z\bar{z} = \|z\|^2$
- $\bar{z} = a - bi - cj - dk$ and $z^{-1} = \bar{z}/\|z\|^2$
- For $z_1 = s_1 + v_1, z_2 = s_2 + v_2$ (where $s_i \in \mathbb{R}$): $z_1 z_2 = s_1 s_2 - v_1 \cdot v_2 + s_1 v_2 + s_2 v_1 + v_1 \times v_2$ where last term is sum of all components of cross product vector

Rotating a point $p = \begin{bmatrix} x & y & z \end{bmatrix}^T$ around axis $u = \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix}$ by angle θ:

- Convert p to quaternion $p_Q = xi + yj + zk$
- Convert u to quaternion $q'' = u_1 i + u_2 j + u_3 k$, normalize $q' = q''/\|q''\|$
- Rotate quaternion $q = \cos(\theta/2) + q' \sin(\theta/2)$ and $q^{-1} = \cos(\theta/2) - q' \sin(\theta/2)$
- Rotated point $p' = qpq^{-1}$. Convert to cartesian.

15 Lighting & Shading

Lighting: Modelling physical interactions between materials and light sources.
Shading: Process of determining color of pixel.

Solid angle: $\Omega = A/r^2$ steradians (where r radius).
Zenith, Azimuth: Point $\omega = (\theta, \phi)$ on unit sphere: $\omega_x = \sin(\theta)\cos(\phi)$, $\omega_y = \sin(\theta)\sin(\phi)$, $\omega_z = \cos(\theta)$.
Energy of photon: $(hc)/\lambda$ J.

Basic quantities of radiometry

Flux: $\Phi(A)$ total amount of energy passing through surface or space per unit time [J/s = W].
Luminous flux: $\int P(\lambda)V(\lambda)\,d\lambda$. Perceived power of light weighted by human sensitiv. [lumen].
Irradiance: $E(x) = d\Phi(A)/dA(x)$ flux per unit area arriving at surface [W/m^2].
Radiosity: $B(x) = d\Phi(A)/dA(x)$ flux per unit area leaving a surface [W/m^2].
Radiant / Luminous Intensity: $I(\omega) = d\Phi/d\omega$ outgoing flux per solid angle [W/sr], $\Phi = \int_{S^2} I(\omega)\,d\omega$.
Radiance: $L(x,\omega) = dI(\omega)/dA(x) = d^2\Phi(A)/(d\omega\, dA(x)\cos\theta)$ flux per solid angle per perp. area.

Lambert's cosine law: Irradiance at surface is proportional to cosine of angle of incident light and surface normal: $E = (\Phi/A)\cos\theta$.
BRDF: $f_r(x,\omega_i,\omega_r) = \frac{dL_r(x,\omega_r)}{dE_i(x,\omega_i)} = \frac{dL_r(x,\omega_r)}{L_i(x,\omega_i)\cos\theta_i\, d\omega_i}$.
Reflection equation: $\int_{H^2} f_r(x,\omega_i,\omega_r)L_i(x,\omega_i)\cos\theta_i\, d\omega_i = L_r(x,\omega_r)$.
Types of reflections: Ideal specular (perfect mirror), ideal diffuse (uniform reflection all directions), Glossy specular (majority light distributed in reflection direction), retro-reflective (reflects light back towards source).
Attenuation: $f_{\text{att}} = 1/d_L^2$ due to spatial radiation; local illumination only considers direct light, global illumination also considers indirect light.

Phong Illumination Model

Approximate specular reflection by cosine powers:

$$I_\lambda = I_{a\lambda}k_a O_{d\lambda} + f_{\text{att}} I_{p\lambda}\left[k_d O_{d\lambda}(N \cdot L) + k_s (R \cdot V)^n\right]$$

I_a ambient light intensity, k_a ambient light coefficient, I_p directed light source intensity, k_d diffuse reflection coefficient, $\theta \in [0, \pi/2]$ angle surface normal N and light source vector L, attenuation factor f_{att}, $O_{d\lambda}$ value of spectrum of object color at the point λ, R is L reflected along the normal N ($R = 2N(N \cdot L) - L$), V is direction towards viewer, n determines shape of highlight.

16 Shading models

Flat shading: one color per primitive/polygon.

Gouraud Shading

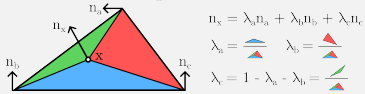
Linear interpolation of vertex intensities.

- Calculate face normals
- Calculate vertex normals by averaging (weighted by angle)
- Evaluate illumination model for each vertex
- Interpolate vertex colors bilinearly on the current scan line

Problems with scan line interpolation are perspective distortion, orientation dependence, shared vertices, incorrect highlights. Quality depends on size of polygons.

Phong Shading

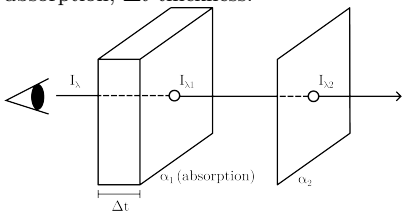
Linear interpolation of vertex normals, barycentric interpolation of normals on triangles.



Properties: Lagrange ($x = a \Rightarrow n_x = n_a$), Partition of unity ($\lambda_a + \lambda_b + \lambda_c = 1$), Reproduction ($\lambda_a a + \lambda_b b + \lambda_c c = x$).
Problems: normal not defined/representative.

Flat, Gouraud in screen space, Phong in obj. space, e.g. to get Gouraud shading we can move the code from Phong shading from fragment shader to vertex shader (only calculate for each vertex, then interpolate).

Transparency: (2 obj., P_1 & P_2). Perceived intensity $I_\lambda = I'_{\lambda 1} + I'_{\lambda 2}$ where $I'_{\lambda 1}$ is emission of P_1 and $I'_{\lambda 2}$ is intensity filtered by P_1. We model it as follows: $I_\lambda = I_{\lambda 1}\alpha_1\Delta t + I_{\lambda 2}e^{-\alpha_1\Delta t}$ where α absorption, Δt thickness.



Linearization: $I_\lambda = I_{\lambda 1}\alpha_1\Delta t + I_{\lambda 2}(1 - \alpha_1\Delta t)$. If last object, set $\Delta t = 1$. Problem: rendering order, sorted traversal of polygons and back-to-front rendering.
Back-to-front order not always clear, resort to depth peeling, multiple passes, each pass renders next closest fragment. We need more advanced lighting models for metal objects (Cook-Torrence), replacing specular term, with reflection from micro facets, self-shadowing...

17 Geometry and Textures

Considerations: Storage, acquisition of shapes, creation of shapes, editing shapes, rendering shapes.

Explicit repr. can easily model complex shapes, and sample points, but take lots of storage.

- Subdivision surface: define surfaces by primitives, use recursive algorithm for refining
- Point set surfaces: store points as surface
- Polygonal meshes: explicit set of vertices with position, possibly with additional information. Intersection of polygons: either empty, vertex, edge.
- Triangle meshes, NURBS, etc.

Implicit repr. easily test inside/outside, compact storage, but sampling expensive, hard to model.

- Algebraic surfaces, constructive solid geometry, level set methods, blobby surfaces, fractals

Need to store textures as bitmaps, hence param. complex surfaces.

Manifolds: surface homeomorphic to disk, closed manifolds divide space into two. In manifold mesh there are at most two faces sharing an edge and each vertex has a 1-connected ring of faces around it (or 1-connected half-ring (i.e. boundary)).

Mesh data structures: Locations, how vertices are connected, attributes such as normals, color etc. Must support rendering, geometry queries, modifications. E.g. **Triangle list** (list of 3 points, redundant, e.g. STL) or **Indexed Face set** (array of vertices + list of indices, e.g. OBJ, OFF, WRL, costly queries, modifications).

Texture mapping

Enhance details without additional geom. complexity. Map Texture (u, v) coords to geom. (x, y, z) coords. Issues: aliasing, level-of-detail, (e.g. sphere mapping: $(u, v) \rightarrow (\sin u \sin v, \cos v, \cos u \sin v)$). We want low-distortion, bijective mapping, efficiency.

Bandlimiting: Restrict amplitude of spectrum to zero for frequency beyond cut-off frequency.

Anti-aliasing filters:

- Gaussian (low-pass, separable). Closer pixels have higher weight
- Sinc $S_{\omega_c}(x, y) = (\omega_c \operatorname{sinc}((\omega_c x)/\pi))/\pi \cdot (\omega_c \operatorname{sinc}((\omega_c y)/\pi))/\pi$ ideal low-pass filter, IIR filter. ω_c cutoff freq. Hard to implement, has infinite impulse response.
- B-Spline: Allow for locally adaptive smoothing, adapt size of kernel to signal chars.

Magnification: for pixels mapping to area larger than pixel (jaggies), use bilinear interpolation.

Mipmapping: Store texture at multiple resolutions, choose based on projected size of triangle (far away $\rightarrow$ lower res, interpol. possible), texels in larger level of mipmap hierarchy represent larger regions of texture (e.g. we halve resolution at each level). Avoids aliasing, improves efficiency, higher storage overhead. (minification)

Supersampling: Multiple color samples per pixel, final color is average (uniform, jittering, stochastic, Poisson).

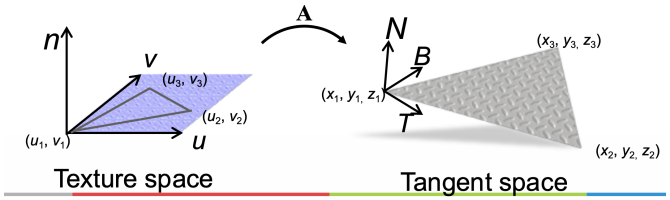
Texture maps

Light map: Simulate effect of local light source (precomputed, dynamically adapted).
Environment map: Mirror environment with imaginary sphere or cube for reflective objects.
Bump map: Perturb surface normal according to texture, represents small-scale geometry. Limitations: no bumps on silhouette, no self-occlusions, no self-shadowing. Height stored as grayscale textures.
Displacement mapping: Compared to bump map, displaces geometry, uses height map to displace points along surface normal.

Procedural texture: Generate texture from noise (Perlin, Gabor) from Gaussian pyramid of noise and summing layers with weights.

Perspective interpolation in world-space can yield non-linear variation in screen-space. Optimal resampling filter is hence spatially variant. To map texture coordinates (u, v) to tangent space we use the matrix A:

$$\begin{bmatrix} | & | \\ \vec{T} & \vec{B} \\ | & | \end{bmatrix} = \mathbf{A} = \begin{bmatrix} x_2 & x_3 \\ y_2 & y_3 \\ z_2 & z_3 \end{bmatrix} \begin{bmatrix} u_2 & u_3 \\ v_2 & v_3 \end{bmatrix}^{-1}$$

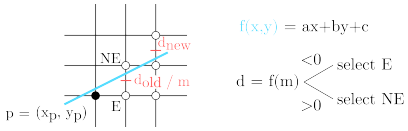


	Primal		Dual
	Triangles	Rectangles	
Approx.	Loop	Catmull-Clark	Doo-Sabin
Interpol.	Butterfly	Kobbelt	

18 Scan Conversion (Rasterization)

Generate discrete pixel values - approximate with finite amount.

Bresenham line: choose closest pixel at each intersection. Fast decision, criterion based on midpoint m and intersection point q. After computing first value d, we only need addition. E: $d_{\text{new}} = d_{\text{old}} + a$, NE: $d_{\text{new}} = d_{\text{old}} + a + b$, $d_0 = a + b/2$, use $a = \Delta y, b = -\Delta x$.



Scan conversion of polygons:

- Calculate intersections on scan line
- Sort intersection points by ascending x-coords.
- Fill spans between two consecutive intersection points if parity in interval is odd (so inside)

19 Bézier Curves & Splines

Bézier Curves

$\mathbf{x}(t) = \mathbf{b}_0 B_0^n(t) + \cdots + \mathbf{b}_n B_n^n(t)$ where B_i^n are the Bernstein polyn. ($\binom{n}{i} = \frac{n!}{i!(n-i)!}$): $B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}$ (partition of unity, pos. definite, recursion, symmetry). Coefficients $\mathbf{b}_0, \ldots, \mathbf{b}_n$ are control points and define the control polygon. Degree is highest order, order is deg. + 1. Bézier curves have affine invariance (affine transf. of curve by affine transf. of control pts.), lie in convex hull of control polygon. Since $B_0^n(0) = B_n^n(1) = 1$, they interpolate endpoints. Max. number of intersects of line with curve is $\leq$ intersects with control polygon.

Disadvantages: global support of basis functions, new control pts yields higher degree, C^r continuity between segments of Bézier curves. **de Casteljau:** Compute a triangular representation, successively interpolate (corner cutting):

$$b_0^1(t) = (1-t)b_0 + tb_1, \quad b_1^1(t) = (1-t)b_1 + tb_2, \quad b_0^2(t) = (1-t)b_0^1(t) + tb_1^1(t)$$

B-Spline functions

B-Spline curve $\mathbf{s}(u)$ built from piecewise polyn. bases $s(u) = \sum_{i=0}^{k} \mathbf{d}_i N_i^n(u)$. Coefficients $\mathbf{d}_i$ are de Boor points. Bases are piecewise, recursively defined polyn. over sequence of knots $u_0 < u_1 < u_2 < \ldots$ defined by knot vector $T = [u_0, \ldots, u_{k+n+1}]$.

$$N_i^n(u) = N_i^{n-1}(u) \frac{u - u_i}{u_{i+n} - u_i} + N_{i+1}^{n-1}(u) \frac{u_{i+n+1} - u}{u_{i+n+1} - u_{i+1}}$$

$$N_i^0(u) = \begin{cases} 1 & u \in [u_i, u_{i+1}] \\ 0 & \text{else} \end{cases}$$

Partition of unity $\sum_i^k N_i^n(u) = 1$, positivity $N_i^n(u) \geq 0$, compact support $N_i^n(u) = 0$ for $u \notin [u_i, u_{i+n+1}]$, continuity N_i^n is $(n-1)$ times differentiable, local control, affine invariant.

B

-spline of degree n and $k+1$ = de Boor point consequently requires $n+k+2$ =knots. Denk nach David

Example: $N_i^1(u) = \begin{cases} \frac{u-u_i}{u_{i+1}-u_i} & u \in [u_i, u_{i+1}] \\ \frac{u_{i+2}-u}{u_{i+2}-u_{i+1}} & u \in [u_{i+1}, u_{i+2}] \end{cases}$

de Boor algorithm: generalizes de Casteljau, evaluate B-spline of degree n at u, set $d_i^0 = d_i$, finally $d_0^n = s(u)$:

$$d_i^k = (1-a_i^k)d_i^{k-1} + a_i^k d_{i+1}^{k-1}, \quad a_i^k = \frac{u-u_{i+k-1}}{u_{i+n}-u_{i+k-1}}$$

Note that a_i^k vanishes outside of $[u_{i+k-1}, u_{i+n}]$.

20 Subdivision Surfaces

Generalization of spline curves/surfaces allowing arbitrary control meshes using successive refinement (subdivision), converging to smooth limit surfaces, connecting splines and meshes.

- **Interpolating** (new points are interpolation of control points) vs. **Approximating** (control points moved, new points added in-between)
- **Primal** (faces are split) vs. **Dual** (vertices are split)

Loop subdivision (triangular meshes).

21 Visibility and Shadows

Painter's algorithm: Render objects from furthest to nearest. Issues with cyclic overlaps/intersections.
Z-Buffer: store depth to nearest obj for each px. Initialize to ∞, then iter. over polygons and update z-buffer. Limited resolution, non-linear, place far from camera.
Planar shadows: draw projection of obj. on ground. No self-shadows, no shadows on other objects, curved surfaces.
Projective texture shadows: Separate obstacles and receivers, compute b/w img. of obstacles from light, use as projective texture. Need to specify obstacle & receiver, no self-shadows.

Shadow maps

Compute depths from light & camera. For each px on camera plane, compute point in world coords., project onto light plane, compare $d(\mathbf{x}_L)$ (dist. from light source to projected point on light plane) to z_L (dist light source to x_L). If $d(\mathbf{x}_L) < z_L$, then x in shadow. Bias needed to avoid self-shadowing ($d(\mathbf{x}_L)+b < z_L$). Point to shadow can be outside field of view (use cubical shadow map or spot lights). Aliasing due to undersampling shadow map (filter result of depth test).

Shadow volumes: Explicitly represent volume of space in shadow. If polygon in volume, it is in shadow. Shoot ray from camera, increment each time boundary of shadow volume is intersected. If counter > 0, primitive is in shadow. Optimization: use silhouette edges only. Introduces a lot of new geometry, expensive to rasterize long skinny triangles. Objects must be watertight for silhouette optimizations. Rasterization of polygons must not overlap and not have a gap.

22 Ray Tracing

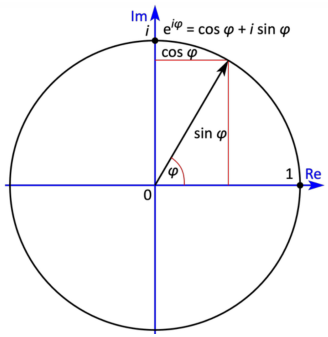
Send rays into scene to determine color of px. On obj. hit, send multiple rays (diffused, reflected, refracted) further until we hit light source, or reach some amount of bounces. Figure out whether point in shadow by shooting rays to all light sources. For anti-aliasing, use multiple rays per px. (Pipeline: Ray Generation, Intersection, Shading)

Ray-surface intersections: Ray equation $\mathbf{r}(t) = \mathbf{o}+t\mathbf{d}$. **Sphere:** $\|\mathbf{x}-\mathbf{c}\|^2 - r^2 = 0$ where $\mathbf{x}$ point of interest, $\mathbf{c}$ center, r radius. Solve for t: $\|\mathbf{o}+t\mathbf{d}-\mathbf{c}\|^2 - r^2 = 0$. **Triangle:** Barycentric coords. $\mathbf{x} = s_1\mathbf{p}_1 + s_2\mathbf{p}_2 + s_3\mathbf{p}_3$. Intersect: $(\mathbf{o}+t\mathbf{d}-\mathbf{p}_1)\cdot\mathbf{n} = 0$. Using $\mathbf{n} = (\mathbf{p}_2-\mathbf{p}_1)\times(\mathbf{p}_3-\mathbf{p}_1)$, we get $t = -((\mathbf{o}-\mathbf{p}_1)\cdot\mathbf{n})/(\mathbf{d}\cdot\mathbf{n})$. Compute s_i, test whether $s_1+s_2+s_3 = 1$ and $0 \le s_i \le 1$.
Ray-tracing shading extensions: Refraction, mult. lights, area lights for soft shadows, motion blur (sample objs, intersect in time), depth of field, cost: $O(N_xN_yN_o) = \#\text{px}\cdot\#\text{objects}$.
Accelerate: introduce uniform grids or space partitioning trees.
Uniform grids: Preprocess: compute bounding box, set grid res., rasterize objects, store refs. to objects. Traversal: incrementally rasterize rays - stop at intersection. Fast & easy, but non-adaptive to scene geometry.
Space partitioning trees: octree, kd-tree, bsp-tree, bounding volume hierarchies (BVH).
Notes:

- Model matrix: from object space to world coordinates, View matrix: from world coordinates to camera coordinates, Projection matrix: from camera coordinates to screen space
- All 3D rot. matrices R have property $R^{-1} = R^T$
- Bézier curves are special cases of B-spline curves
- Color buffer is updated only when depth test passed
- Radiance is constant along a ray (in a vacuum)
- Pinhole camera measures radiance
- $f_r(\omega_i, \omega_o) = f_r(\omega_o, \omega_i)$ is true for any valid BRDF function
- If BRDF satisfies $\int_\Omega f_r(\omega_i, \omega_o)\, d\omega_i = 1$, all incoming energy is reflected
- For a perfect mirror material, $f_r(\omega_i, \omega_o)$ is non-zero iff ω_i is reflection vec. of ω_o against surface normal
- Due to persp. proj., barycentric coords. of values on a triangle of different depths are not an affine function of screen space positions

23 Miscallaneous

- $e^{i\pi} + 1 = 0$
- $e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$
- $e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$
- $e^{-i2\pi ux} = \cos(-2\pi ux) + i\sin(-2\pi ux)$
- $\cos(2\pi ux) = \frac{e^{i2\pi ux}+e^{-i2\pi ux}}{2}$
- $\sin(2\pi ux) = \frac{e^{i2\pi ux}-e^{-i2\pi ux}}{2i}$



positive θ is counter-clockwise rotation.

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2x2 matrix inverse:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \implies A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

angle between two vectors:

$$\theta = \arccos\left(\frac{\mathbf{u}\cdot\mathbf{v}}{|\mathbf{u}|\cdot|\mathbf{v}|}\right)$$

To reflect a vector a across another vector b:

$$\mathbf{a}_{reflected} = 2\frac{\mathbf{a}\cdot\mathbf{b}}{\mathbf{b}\cdot\mathbf{b}}\mathbf{b} - \mathbf{a}$$